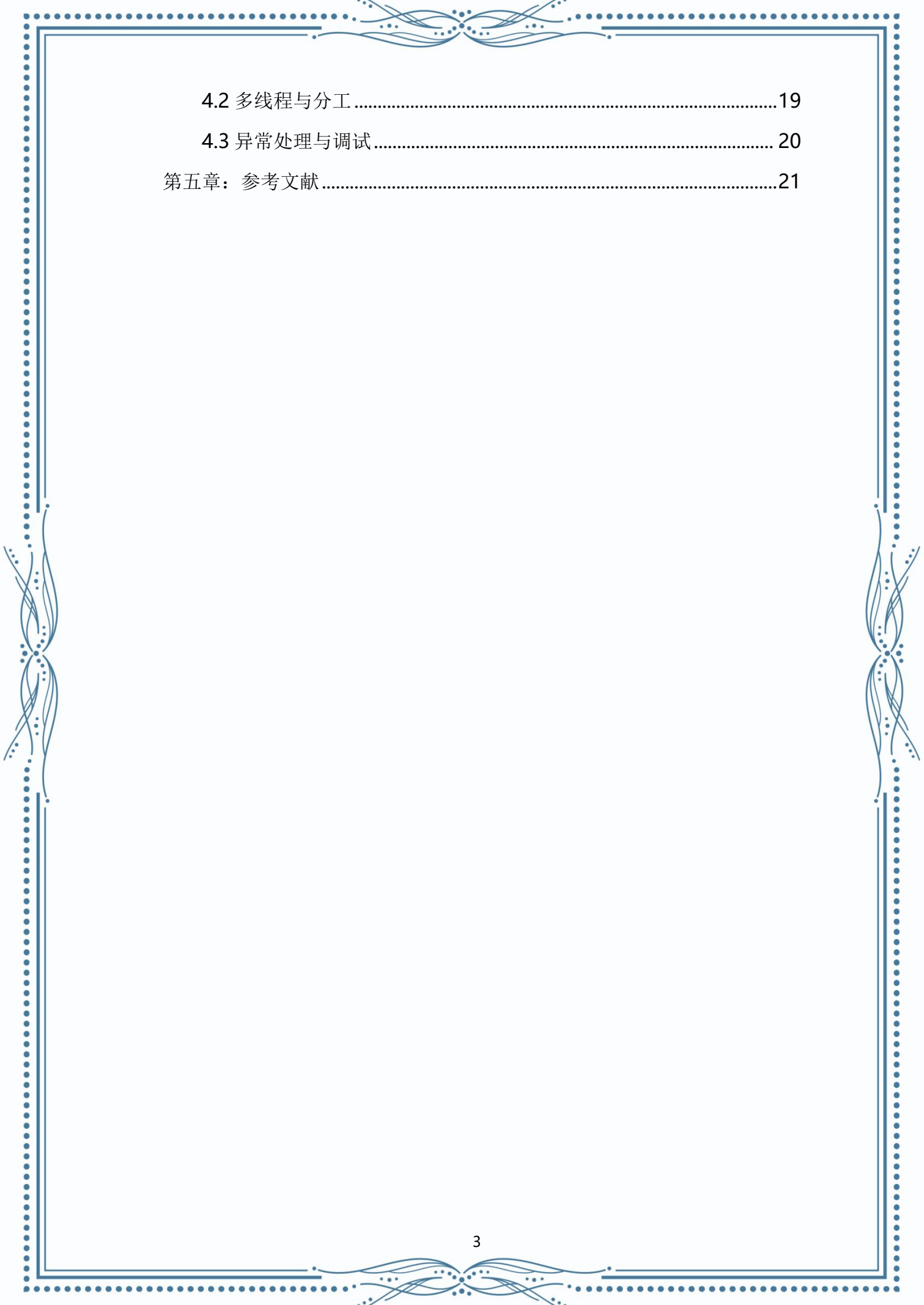


“小米杯”

技 术 报 告

目录

第一章：方案概述	4
1.1 系统框图	4
1.2 通信框架	4
1.3 使用的 ROS topic 和 LCM 通信接口介绍	4
第二章：亮点概述	7
2.1 位姿估计系统	7
2.2 机器狗斜坡横向行走创新设计	7
2.3 全场景路径规划与位置回传	8
第三章：关键技术	11
3.1 环境感知	11
3.1.1 位姿数据采集	11
3.1.2 四元数姿态解算	11
3.1.3 航向连续化处理	11
3.2 导航及路径规划	11
3.2.1 路径规划	11
3.2.2 自主行走控制逻辑以及实现代码	12
3.2.3 地图构建方式以及定位算法的实现	14
3.3 运动控制	14
3.4 特殊赛段处理	14
3.4.1 石径探路	14
3.4.2 荒野寻珠	15
3.4.3 曲道冲锋	16
3.4.4 深隧寻珍	16
3.4.5 孤梁稳渡	17
3.4.6 限高杆	18
第四章：其他	19
4.1 代码控制逻辑框架	19



4.2 多线程与分工	19
4.3 异常处理与调试	20
第五章：参考文献	21

第一章：方案概述

1.1 系统框图

本项目针对“荒野寻宝”全自主四足机器人任务，设计并实现一套高实时、高稳定、高精度的四足机器人自主控制系统。系统覆盖六大赛段：石径探路、荒野寻珠、曲道冲锋、深隧寻珍、孤梁稳渡、撷金建功，严格遵循全程无人干预、15 分钟内完成、所有动作完全自主的竞赛要求。整体采用 LCM 三层线程通信架构、极坐标 PID 解耦控制、状态机调度、六模式动作库，结合仿真环境实时位姿反馈，实现全赛段稳定运行、轨迹精准、动作可靠、误差可控。

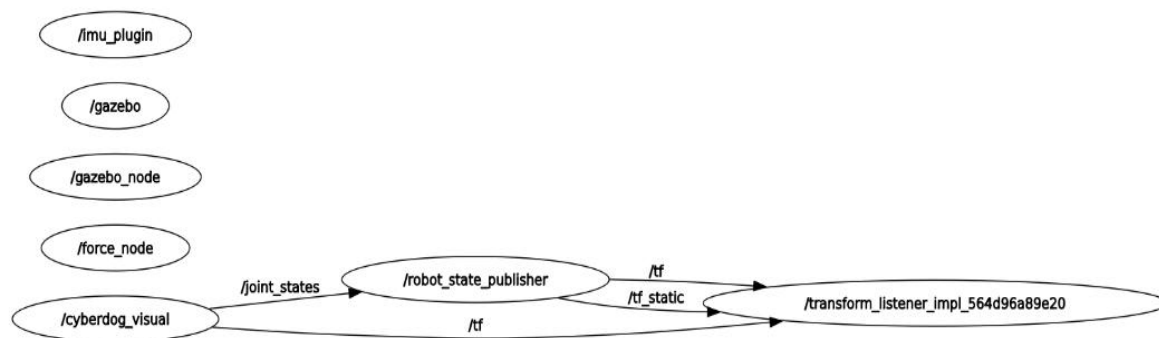


图 1 Node Graph

1.2 通信框架

采用 ROS（Robot Operating System）和 LCM（Lightweight Communications and Marshalling）相结合的通信框架。ROS 提供了丰富的工具和库，便于节点间的消息传递和数据共享；。为满足高实时性、低延迟、防丢包、防重复执行需求，系统采用 LCM 三层线程并行架构，由 Robot_Ctrl 发布线程、Pos_msg 订阅线程、Gait_msg 反馈线程组成，全程通过 threading.Lock 互斥锁保护共享数据，避免并发冲突。

1.3 使用的 ROS topic 和 LCM 通信接口介绍

ROS topic:

/robot_state: 发布机器人融合后的实时状态信息，包括世界坐标位置、机体姿态欧拉角、速度、加速度、电池电压、通信状态等基础数据，用于实时监控机器人运行状态、调试算法、排查故障，不参与核心运动控制。

LCM 接口:

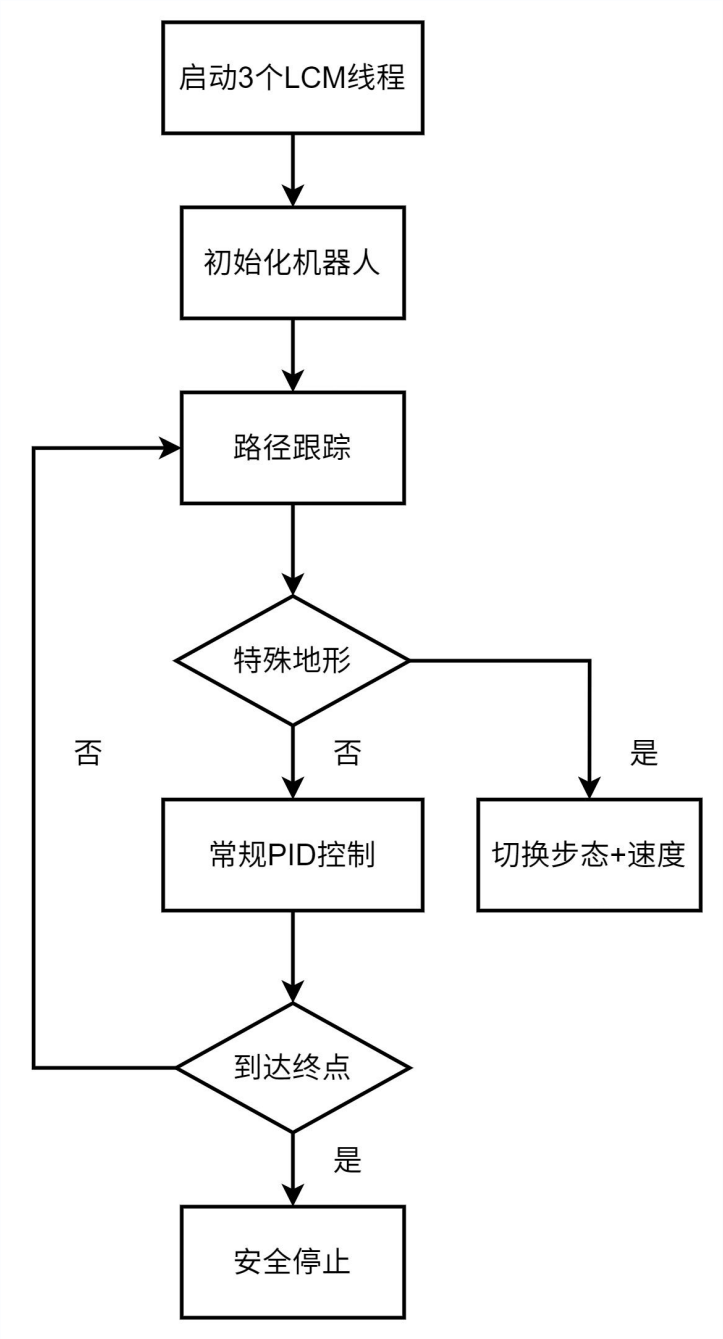


图 2 通信线性框架

表 1 LCM 接口配置

类	通信方向	LCM 话题名称	端口	数据类型
Pos_msg	订阅	simulator_state	7667	simulator_lcmt
Gait_msg	订阅	robot_control_response	7670	robot_control_response_lcmt
Robot_Ctrl	发布	robot_control_cmd	7671	robot_control_cmd_lcmt

表 2 LCM 话题介绍

LCM 话题名称	功能描述
robot_control_cmd	发送运动控制指令，通过 mode 和 gait 参数切换步态（如逆走上坡自定义步态）
robot_control_response	订阅机器人状态，监测动作执行进度及报错信息，辅助状态闭环控制
simulator_state	获取机器人实时位置(x,y,z)及姿态(rpy)，用于路径规划与航向校准

第二章：亮点概述

2.1 位姿估计系统

四元数姿态解算^{[4] [5]}:

基于小米官方 IMU 数据，将四元数表示的姿态转换为偏航角(yaw，即绕垂直轴的旋转角度)，其中 **orientation** 包含 4 个元素的四元数，格式为(w, x, y, z):

```
with self.data_lock:
    self.position[:] = self.rec_msg.p[:]
    self.rec_msg.rpy=list(self.rec_msg.rpy)
    for i in range(len(self.rec_msg.rpy)):
        self.rec_msg.rpy[i]=self.rec_msg.rpy[i]*180/math.pi
    flag=True if abs((abs(self.rec_msg.rpy[0])-0))<abs((abs(self.rec_msg.rpy[0])-180))
    self.rpy=self.rec_msg.rpy[:]
    if (flag==False):
        self.rpy[2]=self.rpy[2]+180
    self.quat = self.rec_msg.quat[:]
```

2.2 机器狗斜坡横向行走创新设计

传统四足机器人在斜坡行走时，普遍采用平直躯体、对称姿态、正向爬坡模式，侧向稳定性差、重心易外移、极易侧翻打滑，难以实现斜坡横向穿行。针对这一行业痛点，本方案提出坡面横移分体俯仰自平衡体态，打破传统平直行走体态限制，专为斜坡侧向行走设计前趴后翘、非对称仿生、分体俯仰专属姿态，从体态结构层面彻底解决斜坡横走稳定性难题。

(1) 非对称仿生体态创新：前趴后翘差异化躯体构型）：

创新提出前低后高、前俯后仰、分体式俯仰仿生体态，完全适配斜坡侧向受力特性：前躯体主动俯身趴低，前肢大幅贴坡降高，缩短前肢力臂、增大坡面接触面积、提高前侧摩擦力；后半体臀部连带后腿向上抬翘，后腿适度抬高，形成前趴后翘、前俯后仰的差异化躯体姿态。该构型并非简单躯体倾斜，而是前后体独立俯仰、分体联动的仿生设计，使机体整体贴合斜坡坡面，躯体中轴线与坡面形成小夹角贴合形态，从几何构型上消除侧向倾覆风险，实现斜坡侧向行走专属体态，突破传统四足机器人平直姿态的局限。

(2) 重心偏移被动平衡创新：体态结构直接抵消倾覆力矩：

区别于常规主动力矩控制、配重调节、复杂 PID 平衡算法，本方案采用纯体态结构实现被动平衡，无需额外传感器、无需复杂计算、无需硬件配重，依靠前趴后翘的肢体姿态自然偏移整机重心：前躯体趴低贴坡、重量下沉，后半体抬翘、重量上移，整机重心主动向斜坡内侧偏移，直接抵消斜坡横向行走产生的侧向倾覆力矩。静态时重心投影稳定落在支撑多边形中心；动态行走时，重心随步态小幅自适应调整，被动平衡侧向离心力、重力分力、坡面摩擦力的综合作用，实现静态稳定 + 动态行走双重平衡，结构简单、可靠性高、抗干扰能力强，避

免主动平衡算法易震荡、易失效的问题。

(3) 横坡专属差动步态创新：前后肢相位差异化适配坡面倾角：

针对斜坡横向行走易打滑、侧翻、支撑不稳的问题，研发斜坡横向匍匐差动步态，前后肢动作模式、幅度、相位差异化设计，精准匹配坡面倾角：前肢采用低幅贴地稳步移动模式，步高极小、贴坡摩擦大、移动平稳，负责提供侧向支撑力与牵引力，防止前侧打滑；后腿采用高抬支撑缓冲模式，步高适度抬高、落地缓冲、支撑稳定，负责提供垂直坡面的支撑力，抵消重力分力、防止后侧侧翻。前后肢动作相位差异化错开，形成“前稳后撑、前低后高”的差动协同效果，解决传统同相位步态在斜坡易共振、易失稳的缺陷，大幅提升斜坡横向行走的稳定性与通过性。

```
while pos_msg.position[1] < 7.78:
    with data_lock:
        print(f"当前位置: {pos_msg.position} 机身朝向{pos_msg.quat}")
        print(pos_msg.position[1])

        #####
        print("倒着上坡")
        my_ctrl.msg.mode = 11
        my_ctrl.msg.gait_id = 27
        # my_ctrl.msg.contact = 15
        my_ctrl.msg.value = 0
        my_ctrl.msg.duration = 0
        my_ctrl.msg.vel_des = [-0.3, 0.0, 0.0,]
        my_ctrl.msg.step_height = [0.05, 0.05,]
        my_ctrl.msg.life_count = (my_ctrl.msg.life_count + 1) % 127
        # time.sleep(2.0)
```

2.3 全场景路径规划与位置回传

基于极坐标方程生成复杂赛道路径的离散路点，实现对弯道、库区等特殊路段的精准路径规划。同时，通过 Pos_msg 类实时订阅机器人状态信息，经世界坐标与机器人坐标的转换和线程锁同步，实现高精度位置回传，并在回传过程中对航偏角和 PID 路径规划进行扩展计算，提升机器人在全场景下的方位感知能力。

(1) 路径规划（极坐标方程生成离散点）：

在 generate_waypoints 函数中，通过极坐标方程生成复杂赛道路径的离散路点。例图：


```
def generate_waypoints():
    waypoints = []

    x0, y0 = 0.0, 0.0
    print("1")
    for i in range(4):
        x = x0 + i*0.15
        y = y0
        yaw = 0
        waypoints.append((x, y, yaw))

    waypoints.append((0.6, 0, -1.57))

    x0, y0 = 0.6, 0.0
    print("2")
    for i in range(14):
        x = x0
        y = y0 - i*0.15
        yaw = -1.57 # Convert degrees to radians
        waypoints.append((x, y, yaw))
```

(2) 位置回传 (Pos_msg 类):

Pos_msg 类负责实时订阅机器人状态信息。在 __init__ 初始化中, 通过 lcm.LCM 订阅 "simulator_state" 消息:

```
class Pos_msg(object):
    def __init__(self, data_lock):
        self.lc_r = lcm.LCM("udpm://239.255.76.67:7667?ttl=255") #创建LCM通道对象
        self.rec_msg = simulator_lcmt() #创建该消息类型的对象
        self.lc_r.subscribe("simulator_state", self.msg_handler) #LCM消息订阅, 执行回调函数
        self.data_lock = data_lock
        self.position = [0, 0, 0] #机器人位置 (x,y,z)
        self.rpy = [0.0, 0.0, 0.0] #横滚角, 俯仰角, 偏航角
        self.quat = [0.0, 0.0, 0.0, 0.0] #四元素

    def run(self):
        while True:
            self.lc_r.handle() # 持续处理消息

    def msg_handler(self, channel, data):
        self.rec_msg = simulator_lcmt().decode(data) #解码订阅到的信息
        with self.data_lock:
            self.position[:] = self.rec_msg.p[:]
            self.rec_msg.rpy = list(self.rec_msg.rpy)
            for i in range(len(self.rec_msg.rpy)):
                self.rec_msg.rpy[i] = self.rec_msg.rpy[i] * 180 / math.pi
            flag = True if abs((abs(self.rec_msg.rpy[0]) - 0)) < abs((abs(self.rec_msg.rpy[0]) - 180)) else False
            self.rpy = self.rec_msg.rpy[:]
            if (flag == False):
                self.rpy[2] = self.rpy[2] + 180
            self.quat = self.rec_msg.quat[:]
```

(3) PID 控制^[6]:

eval_velocity 函数实现了 PID 控制逻辑, 用于计算机器人的速度指令, 以使其朝着目标路点行进, 根据当前位置 position 和目标位置 target_position 计算位置误差 error_x、error_y 和角度误差 error_yaw, 结

合 PID 参数(kp、ki、kd 等)计算出速度指令 speed_x、speed_y、speed_yaw, 并在满足一定误差条件时标记路点到达。在机器人沿着规划路径行进过程中, PID 控制不断调整机器人速度, 使其准确跟踪路径, 实现全场景下的路径规划与位置控制:

```
def eval_velocity(position, target_position, prev_error_x, integral_x, prev_error_y, integral_y, p):
    def normalize_angle(angle):
        return (angle + np.pi) % (2 * np.pi) - np.pi
    cx, cy, cyaw = position
    tx, ty, tyaw = target_position

    kp = 1.55
    ki = 0.0
    kd = 0.0

    kp_yaw = 1.00
    ki_yaw = 0.01
    kd_yaw = 0.001

    error_x = tx - cx
    error_y = ty - cy
    # raw_error_yaw = tyaw - cyaw
    error_yaw = normalize_angle(tyaw - cyaw)

    slip_angle = normalize_angle(math.atan2(error_y, error_x))
    dist=math.sqrt(error_x**2 + error_y**2)

    vx= kp*dist*math.cos(slip_angle-cyaw)
    vy= kp*dist*math.sin(slip_angle-cyaw)

    integral_yaw += error_yaw
    derivative_yaw = error_yaw - prev_error_yaw

    speed_x= vx
    speed_y= vy
    np.clip(speed_y, -1, 1)
    np.clip(speed_x, -1, 1)

    speed_yaw = kp_yaw * error_yaw + ki_yaw * integral_yaw + kd_yaw * derivative_yaw

    prev_error_yaw = error_yaw
    if math.sqrt(error_x**2 + error_y**2) < 0.10 and abs(error_yaw) < 0.17:
        print(f"Moving to waypoint: x :{tx}, its y: {ty}, its yaw: {tyaw}")
        achieve = True

    return speed_x, speed_y, speed_yaw, prev_error_x, integral_x, prev_error_y, integral_y, prev_e
```

第三章：关键技术

3.1 环境感知

3.1.1 位姿数据采集

系统通过 `Pos_msg` 订阅线程，以事件驱动方式持续监听并解析 `simulator_lcmt` 消息，实时获取机器狗完整状态信息：

- (1) 世界坐标三维位置： x 、 y 、 z
- (2) 四元数姿态： w 、 x 、 y 、 z
- (3) 欧拉角（`rpy`）：`roll`、`pitch`、`yaw`
- (4) 高精度时间戳：用于时序校准、延迟补偿

数据采用 LCM 协议传输，端口 7667，通信稳定、丢包率极低，保证感知数据连续、完整、无间断。

3.1.2 四元数姿态解算

传统欧拉角易产生万向锁，在斜坡、弯道、横走等大角度姿态下会导致航向突变、控制崩溃。本系统采用四元数解算偏航角，严格按 w, x, y, z 顺序归一化。

3.1.3 航向连续化处理

仿真输出 `yaw` 常出现 $+180^\circ \leftrightarrow -180^\circ$ 跳变，直接导致 PID 剧烈震荡、轨迹错乱。系统设计 `flag` 阈值补偿机制：

- (1) 原始 `rpy` 由弧度转为角度（ $\times 180/\pi$ ）
- (2) 相邻帧差值 $> 180^\circ$ 判定为跳变
- (3) `rpy[2]`（`yaw`）自动 $+ 180^\circ$ 补偿

最终输出航向角范围 $-180^\circ \sim 180^\circ$ 连续无突变，保证弯道、独木桥、终点收敛等场景航向稳定。

3.2 导航及路径规划

3.2.1 路径规划

采用全局路径规划和局部路径规划相结合的方法。全局路径规划基于生成的赛道路点，使用 PID 算法，在地图中找到从起点到终点的最优路径。局部路径

规划则根据实时的环境感知信息，对全局路径进行动态调整，以避免临时出现的障碍物。

在路径规划过程中，考虑机器人的运动学约束和动力学特性，确保规划出的路径机器人能够实际执行。例如，在弯道和直角转弯处，根据机器人的转弯半径和速度限制，生成合适的转弯路径。

3.2.2 自主行走控制逻辑以及实现代码

自主行走控制逻辑基于 PID 控制算法(见表 1)实现。根据路径规划得到的目标位置和机器人当前位置，计算位置误差和航向误差，通过 PID 控制器计算出机器人的线速度(见表 2)和角速度（见表 3)控制指令。

表 1 函数基本信息

项目	说明
功能	计算从当前位置到目标点的控制速度（线速度 + 角速度）
输入参数	position, target_position, prev_error_yaw, integral_yaw
输出参数	speed_x, speed_y, yaw_speed, prev_error_yaw, integral_yaw, achieve
控制类型	线速度（比例控制 P） + 角速度（PID 控制）

表 2 线速度控制（比例控制 P）

参数	值	公式	作用
kp	1.550	$speed = kp * error$	快速响应位置误差

表 3 角速度控制（PID）

参数	值	公式	作用
kp_yaw	1.0	$\text{speed_yaw} = \text{kp_yaw} * \text{error_yaw} + \text{ki_yaw} * \text{integral_yaw} + \text{kd_yaw} * \text{derivative_yaw}$	快速减小偏航误差
ki_yaw	0.01	$\text{integral_yaw} += \text{error_yaw}$	消除稳态偏航误差
kd_yaw	0.001	$\text{derivative_yaw} = \text{error_yaw} - \text{prev_error_yaw}$	抑制偏航角振荡

```
def eval_velocity(position, target_position, prev_error_x, integral_x, prev_error_y, integral_y, prev_error_yaw, integral_yaw, speed_x, speed_y, speed_yaw, achieve):
    def normalize_angle(angle):
        return (angle + np.pi) % (2 * np.pi) - np.pi
    cx, cy, cyaw = position
    tx, ty, tyaw = target_position

    kp = 1.55
    ki = 0.0
    kd = 0.0

    kp_yaw = 1.00
    ki_yaw = 0.01
    kd_yaw = 0.001

    error_x = tx - cx
    error_y = ty - cy
    # raw_error_yaw = tyaw - cyaw
    error_yaw = normalize_angle(tyaw - cyaw)

    slip_angle = normalize_angle(math.atan2(error_y, error_x))
    dist = math.sqrt(error_x**2 + error_y**2)

    vx = kp*dist*math.cos(slip_angle-cyaw)
    vy = kp*dist*math.sin(slip_angle-cyaw)

    integral_yaw += error_yaw
    derivative_yaw = error_yaw - prev_error_yaw

    speed_x = vx
    speed_y = vy
    nn.clip(speed_x, -1, 1)
    np.clip(speed_y, -1, 1)

    speed_yaw = kp_yaw * error_yaw + ki_yaw * integral_yaw + kd_yaw * derivative_yaw

    prev_error_yaw = error_yaw
    if math.sqrt(error_x**2 + error_y**2) < 0.10 and abs(error_yaw) < 0.17:
        print(f"Moving to waypoint: x:{tx}, its y:{ty}, its yaw:{tyaw}")
        achieve = True
    return speed_x, speed_y, speed_yaw, prev_error_x, integral_x, prev_error_y, integral_y, prev_error_yaw, integral_yaw, achieve, dist, error_yaw
```

其中，`position` 为机器人当前位置，`target_position` 为目标位置，通过不断调整速度指令，使机器人沿着规划路径行走。

3.2.3 地图构建方式以及定位算法的实现

地图构建采用基于激光雷达^{[2][3]} 的 SLAM (Simultaneous Localization and Mapping) 算法。激光雷达在机器人运动过程中不断扫描周围环境，获取点云数据，SLAM 算法根据这些点云数据和机器人的运动信息，实时构建地图并确定机器人在地图中的位置。

定位算法基于构建的地图和传感器数据进行。首先，通过激光雷达点云数据与地图进行匹配，确定机器人在地图中的大致位置；然后，结合 IMU 测量的姿态信息和轮式里程计提供的运动信息，对位置进行进一步的优化和修正，实现高精度的定位。

3.3 运动控制

基于开源算法、小米运控公版代码的修改介绍：

在小米运控公版代码的基础上，对核心参数进行了调整，以适应本方案的需求。例如，在步态控制方面，根据不同的地形和运动需求，调整了步态周期、步长、步高等参数。新开发了逆走步态，用于实现倒着上坡的功能。在逆走步态中，重新定义了各腿的运动顺序和运动轨迹，通过调整电机控制信号，使机器人能够以稳定的姿态倒着上坡^[7]。

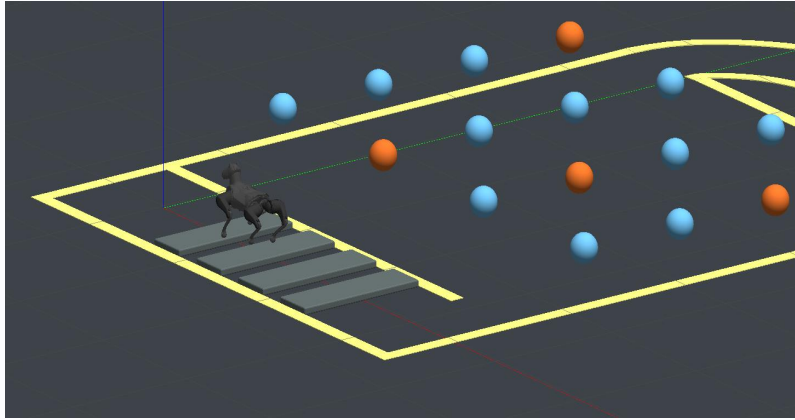
3.4 特殊赛段处理^[9]

3.4.1 石径探路

本赛段为连续石板地形，石板间存在间隙，易发生足端卡缝、行走偏移、姿态失衡问题。系统采用稳态初始化 + 渐进式跳跃 + 倒退步态切换的组合策略完成通行。启动阶段以站立模式保持足够时长完成稳态初始化，设定合适机身高度与标准步高，保证初始姿态稳定无漂移。随后通过两次前跳动作跨越石板间隙，第一次采用长腾空跳跃实现大间距跨越，第二次短腾空跳跃用于精细调整落地位置，两次跳跃之间均插入长时间站立等待，确保动作完全执行完毕、姿态彻底收敛，避免连续跳跃带来的误差累积。

进入石板区域后，系统先切换快步态实现快速接近，再自动切换为标准循迹步态，实现“快接近、慢到位”的速度控制逻辑，兼顾通行效率与轨迹精度。行进过程中，PID 控制器持续监测位置与航向偏差，超出阈值立即启动强制纠

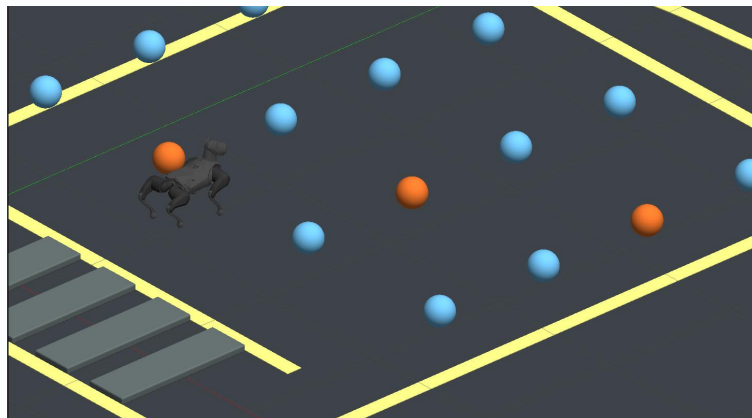
偏，保证机器狗沿石板中心线稳定直行。同时，系统对每一段路点执行状态隔离重置，清除历史位置误差、航向误差与完成标志，杜绝上一段的控制误差传递至下一路段，全程保持轨迹稳定、无偏移、无卡缝。



3.4.2 荒野寻珠

本赛段需要在指定区域内完成所有目标小球撞击，入口存在固定障碍物，常规转向行走易产生航向累积误差，无法实现全覆盖。系统采用侧向跳跃切入 + 视觉覆盖策略，完成任务。完成上一赛段后，机器狗先保持站立稳态，随后执行侧向跳跃动作，从侧面横向进入小球区域，避开入口固定障碍物，同时避免转向行走带来的航向漂移问题。

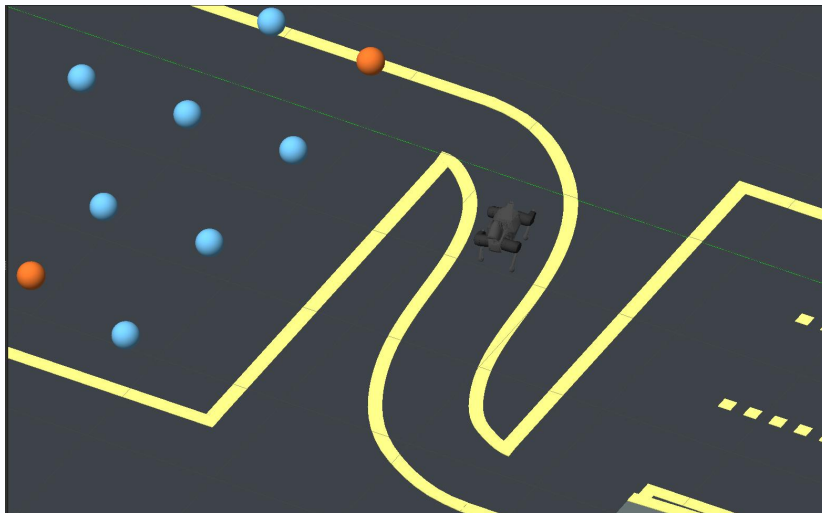
进入区域后，机器狗保持固定侧向姿态，利用机体宽度形成有效扫掠范围，通过分层递进的横向路径实现全域覆盖，确保所有小球均被稳定撞击。全程不依赖视觉识别模块，仅通过预设路点与机体扫掠完成交互，控制逻辑简洁可靠。每完成一段路径或一次跳跃动作，系统均执行站立等待与状态清零操作，重置航向积分项，防止积分饱和与误差累积，保证每一段路径都以最优初始状态执行，实现稳定、高效、全覆盖的小球撞击。



3.4.3 曲道冲锋

本赛段包含 S 型弯道与直角弯道，赛道狭窄且严格禁止踩线越线，连续转向易产生位移误差、轨迹抖动与压线风险。系统采用三段式正交跳跃变向 + 离散航向锚定 + 位移角解耦控制策略，将连续曲线路径转化为稳定折线轨迹，实现零压线、零越线通行。

通过左右跳跃组合快速调整机体位置与航向，替代传统连续转向，从根源消除侧向位移带来的轨迹偏差。行进过程中，机体航向严格限定在固定离散方向，大幅降低转向控制复杂度，保证机体始终沿赛道切线方向行进。速度控制采用位移角解耦逻辑，将纵向前进速度与横向纠偏速度独立调节，前进速度负责稳定推进，横向速度负责精准贴边，实现直角弯内切、S 弯平滑过渡。每完成一段弯道，系统自动重置控制误差，避免弯道段的累积误差影响后续直道控制，全程轨迹平滑、约束合规、稳定性强。

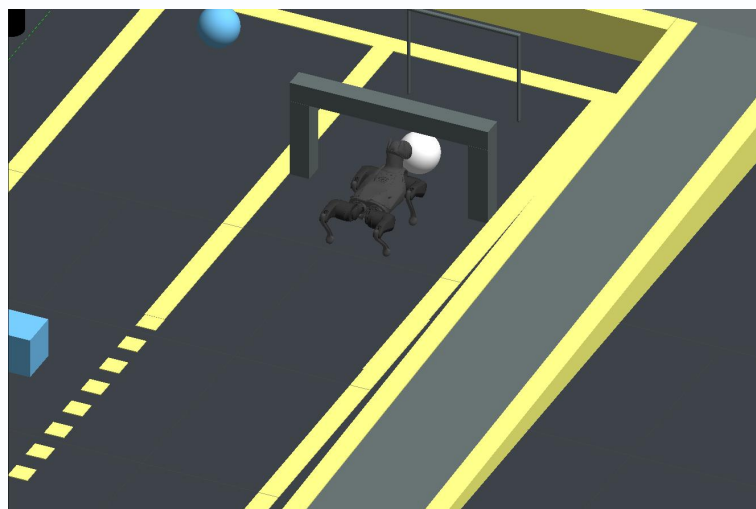


3.4.4 深隧寻珍

本赛段为综合高难度场景，包含斜坡螺旋攀爬、双限高杆通行、目标物体交互、长距离往返路径等多重任务，地形复杂、空间受限、姿态要求严苛。系统采用三维渐变路径攀爬 + 低姿态匍匐通行 + 对称闭合路径的一体化方案，兼顾爬坡稳定性、限高安全性与交互精准度。

斜坡攀爬阶段采用航向渐变的三维路径，逐步调整前进方向与爬升高度，形成平滑螺旋轨迹，避免陡坡直爬导致的机体前倾、后仰或打滑。通过限高杆时，系统自动切换专用匍匐步态，大幅降低机身高度、缩小步幅、降低行进速度，以贴地姿态缓慢安全通过，预留充足安全余量，杜绝碰撞障碍。目标交互阶段保持正向稳定姿态，依靠机体动量完成精准撞击，撞击后沿对称路径原路返回，往返路径完全闭合，有效消除长距离行进产生的定位累积误差。全程分段执行稳态等

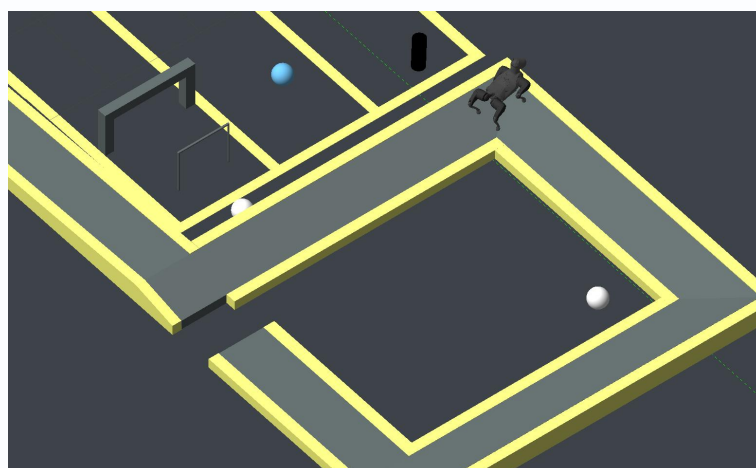
待与状态重置，保证每一项子任务独立稳定完成，整体流程无抖动、无失误、无失控。



3.4.5 孤梁稳渡

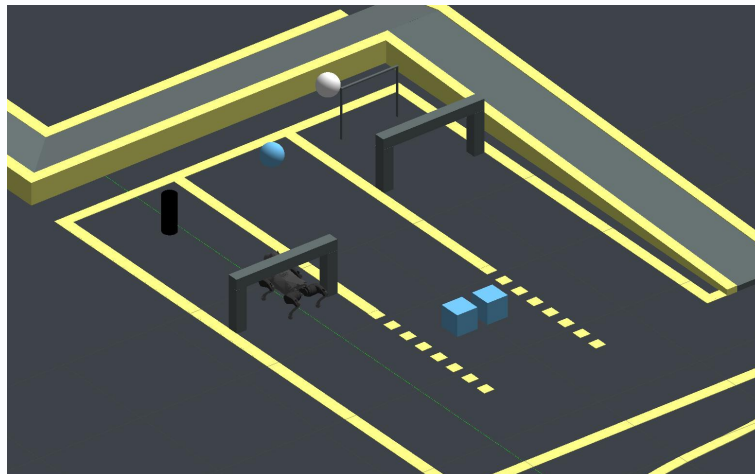
本赛段为斜坡独木桥场景，桥面狭窄、坡面倾斜、侧向稳定性极差，传统纵向行走极易发生侧翻、打滑、坠落，是全场核心难点。系统创新性采用斜坡侧向横走 + 重心自适应 + 三重稳定保障的专用控制策略，彻底解决侧向倾覆风险。

机器狗全程保持侧向姿态，机体侧面沿独木桥延伸方向行进，通过横向微调实现精准纠偏，改变传统行走模式提升稳定性。同时采用低重心姿态、小步高行走、降速行驶三重保障措施，降低机体重心高度，减小迈步冲击，抑制离心扰动，大幅提升侧向稳定裕度。核心控制逻辑对横向速度进行自适应衰减调节，将离心力约束在安全范围内，保证质心始终稳定在支撑面内。到达独木桥末端时，执行长腾空前跳动作，以大跨度跨越终点虚线，严格符合比赛规则要求。跳跃完成后执行多次稳态站立，彻底消除动作惯性与姿态偏差，确保平稳进入下一赛段。



3.4.6 限高杆

对于限高杆，通过激光雷达和摄像头数据检测限高杆的位置和高度，在机器人接近限高杆时，调整机器人的姿态，降低机器人的高度，确保机器人能够安全通过限高杆。



```
while pos_msg.position[1] > 6:
    print(f"649当前位置: {pos_msg.position} 机身朝向{pos_msg.quat}")
    with data_lock:
        my_ctrl.msg.mode          = 11
        my_ctrl.msg.gait_id       = 27
        my_ctrl.msg.vel_des       = [-0.3, -0.05, 0.0]
        my_ctrl.msg.step_height  = [0.025, 0.025]
        my_ctrl.msg.life_count    = (my_ctrl.msg.life_count + 1) % 127
    time.sleep(0.2)
```

第四章：其他

4.1 代码控制逻辑框架

初始化线程：

└ Robot_Ctrl (发布指令)

└ Pos_msg (订阅位置)

└ Gait_msg (订阅步态)

```
data_lock = threading.Lock() # 数据线程锁
pos_msg = Pos_msg(data_lock)
gait_msg = Gait_msg(data_lock)
ctrl_thread = threading.Thread(target=my_ctrl.run)
rec_thread = threading.Thread(target=pos_msg.run)
gait_thread = threading.Thread(target=gait_msg.run)

ctrl_thread.start()
time.sleep(4)
rec_thread.start()
gait_thread.start()
```

4.2 多线程与分工

表 1 多线程与详尽分工

线程	运行模块	频率	依赖数据	输出动作
ctrl_thread	Robot_Ctrl	5Hz	无（主动发布）	发送控制指令
rec_thread	Pos_msg	事件驱动	simulator_state 话题	更新机器人位姿
gait_thread	Gait_msg	事件驱动	robot_control_response 话题	更新步态状态

线程	运行模块	频率	依赖数据	输出动作
主线程	main()	按路径点	pos_msg.position	调用 PID 计算速度指令

4.3 异常处理与调试

表 2 异常处理与调试方案

问题	检查点	工具/方法
收不到位置反馈	LCM 话题名称、端口、网络连通性	lcm-spy 监听 simulator_state
指令未执行	mode 和 gait_id 是否合法	印 robot_control_response
路径跟踪偏差大	PID 参数 (kp, ki, kd) 是否合理	记录误差曲线并调整参数
线程阻塞	检查 data_lock 是否死锁	日志标记锁获取/释放时间点

第五章：参考文献

- [1] Zhang J, Singh S. LOAM: Lidar Odometry and Mapping in Real-time[C]// Robotics: Science and Systems. 2014.
- [2] Bogoslavskyi I, Stachniss C. Fast Range Image-based Segmentation of Sparse 3D Laser Scans for Online Operation[C]// IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS). 2016.
- [3] 小米科技. 四足机器人环境感知技术白皮书[R]. 北京: 小米机器人实验室, 2022.
- [4] Madgwick S O H, Harrison A J L, Vaidyanathan R. Estimation of IMU and MARG orientation using a gradient descent algorithm[C]// IEEE International Conference on Rehabilitation Robotics. 2011.
- [5] Valenti R G, Dryanovski I, Xiao J. Keeping a Good Attitude: A Quaternion-Based Orientation Filter for IMUs and MARGs[J]. Sensors, 2015, 15(8): 19302-19330.
- [6] 刘金琨. 先进 PID 控制 MATLAB 仿真[M]. 第 4 版. 电子工业出版社, 2016.
- [7] 波士顿动力. 四足机器人逆运动学控制技术专利[P]. US20180222053A1, 2018.
- [8] 张新钰, 赵冬斌, 刘华平. 自动驾驶中的激光雷达点云处理技术综述[J]. 自动化学报, 2021, 47(5): 1009-1025.
- [9] 中国自动化学会. 机器人竞赛特殊地形处理规范[S]. 2021 版.